Backpropagation Through a Two-Layer Binary Classifier

A complete symbolic derivation from binary cross-entropy to the first layer

Dr. A. Belcaid

Table of contents

1 Network and notation 2

2 Forward pass: store what backward will need 2

3 Backward pass, one block at a time 3
3.1 Step 0: seed the loss . . . 3
3.2 Step 1: binary cross-entropy sends a message to $\widehat{y}$ . . . 3
3.3 Step 2: the output sigmoid sends a message to z_2 . . . 4
3.4 Step 3: the second affine layer sends three messages . . . 4
3.5 Step 4: the hidden sigmoid sends a message to $\mathbf{z}_1$ . . . 5
3.6 Step 5: the first affine layer reaches the parameters and input . . . 5

4 The complete backward pass 6

5 Shape audit 6

6 Mini-batches 7

7 A final self-check 7

This note is the complete derivation referenced in the backpropagation lecture. Its purpose is to make every gradient message explicit without interrupting the lecture with a long sequence of algebra.

> **!** **The rule used at every node**
>
> A node first **adds all gradient messages received from its children**. It then multiplies that accumulated gradient by the appropriate local derivative and sends the result to each parent.
>
> Receive and add. Transform locally. Send backward.

1 Network and notation

We consider one training example with input $\mathbf{x} \in \mathbb{R}^D$ and binary target $y \in \{0, 1\}$. The hidden layer contains H units. Vectors are column vectors:

$$W_1 \in \mathbb{R}^{H \times D}, \qquad \mathbf{b}_1 \in \mathbb{R}^H, qquad W_2 \in \mathbb{R}^{1 \times H}, \qquad b_2 \in \mathbb{R}.$$

The forward pass is

$$\begin{aligned} \mathbf{z}_1 &= W_1 \mathbf{x} + \mathbf{b}_1, \\ \mathbf{h} &= \sigma(\mathbf{z}_1), \\ z_2 &= W_2 \mathbf{h} + b_2, \\ \hat{y} &= \sigma(z_2), \\ \mathcal{L} &= -\left[y \log \hat{y} + (1 - y) \log(1 - \hat{y}) \right]. \end{aligned}$$

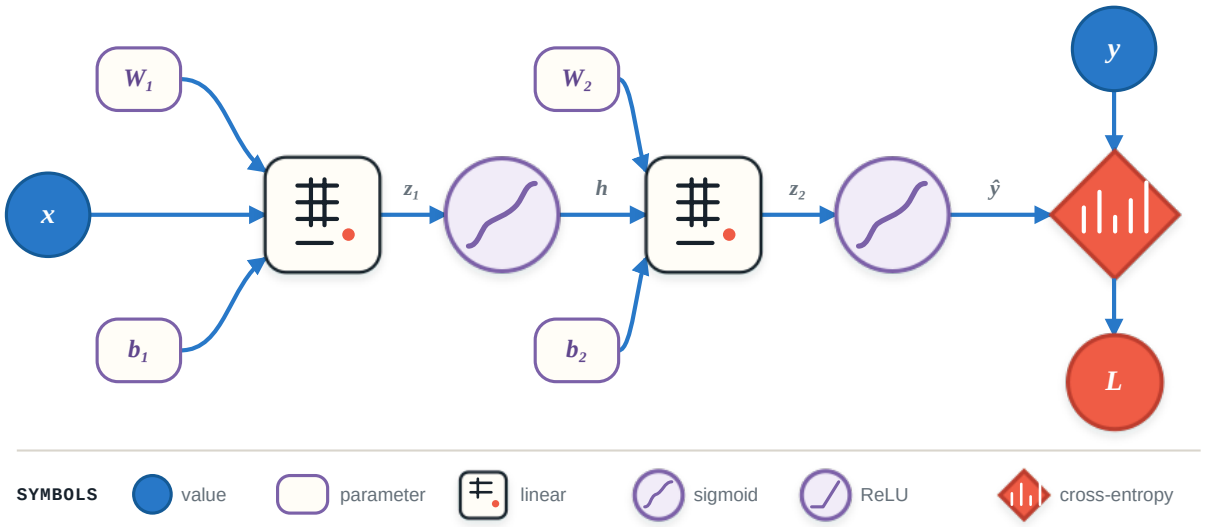


Figure 1: The symbolic computation graph used in the lecture: values are blue circles, parameters are purple rounded rectangles, affine maps are square blocks, sigmoid activations are purple circles, and binary cross-entropy is the coral loss node.

We write

$$\mathbf{g}_v \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{v}}$$

for the gradient message accumulated at a value $\mathbf{v}$. For a scalar, the boldface is omitted.

2 Forward pass: store what backward will need

Backpropagation reuses the values computed during the forward pass. Store

$$\mathbf{x}, quad\mathbf{z}_1, quad\mathbf{h}, quad z_2, quad\hat{y}, quad y.$$

The sigmoid derivative can then be evaluated from its stored output:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

> **i** The backward pass does not repeatedly expand the complete nested formula. Each block differentiates only its own input-output relationship.

3 Backward pass, one block at a time

3.1 Step 0: seed the loss

The loss is the final scalar, so its incoming sensitivity to itself is

$$g_{\mathcal{L}} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1.$$

Every following step has the same structure:

$$\text{outgoing message} = \text{incoming message} \times \text{local derivative}.$$

3.2 Step 1: binary cross-entropy sends a message to $\hat{y}$

The local derivative is

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}.$$

Therefore

$$g_{\hat{y}} = g_{\mathcal{L}} \frac{\partial \mathcal{L}}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}.$$

This expression can become numerically large when $\hat{y}$ is extremely close to zero or one. The next step reveals why software libraries usually combine sigmoid and BCE into one stable operation.

3.3 Step 2: the output sigmoid sends a message to z_2

The sigmoid's local derivative is

$$\frac{\partial \hat{y}}{\partial z_2} = \hat{y}(1-\hat{y}).$$

Multiply the incoming message by this local derivative:

$$\begin{aligned} g_{z_2} &= g_{\hat{y}} \frac{\partial \hat{y}}{\partial z_2} \\ &= \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right) \hat{y}(1-\hat{y}) \\ &= -y(1-\hat{y}) + (1-y)\hat{y} \\ &= \boxed{\hat{y} - y}. \end{aligned}$$

Sigmoid plus BCE is one simple backward message

For binary cross-entropy applied to a sigmoid output,

$$\boxed{g_{z_2} = \hat{y} - y}.$$

This is the prediction error in logit space. Stable implementations compute BCE directly from logits and produce this message without separately forming the potentially unstable fractions above.

3.4 Step 3: the second affine layer sends three messages

The second affine layer is

$$z_2 = W_2 \mathbf{h} + b_2.$$

It receives the scalar message g_{z_2}. The local rule for a linear layer gives

$$\boxed{\frac{\partial \mathcal{L}}{\partial W_2} = g_{z_2} \mathbf{h}^\top} \qquad [1 \times H],$$

$$\boxed{\frac{\partial \mathcal{L}}{\partial b_2} = g_{z_2}} \qquad [\text{scalar}],$$

and the message sent to the hidden activation is

$$\boxed{\mathbf{g}_h = W_2^\top g_{z_2}} \qquad [H \times 1].$$

The transpose is forced by the message-passing interpretation: every hidden activation must receive the contribution carried through its outgoing weight.

3.5 Step 4: the hidden sigmoid sends a message to $\mathbf{z}_1$

The hidden sigmoid acts elementwise:

$$\mathbf{h} = \sigma(\mathbf{z}_1).$$

Its local derivative is the vector

$$\mathbf{h} \odot (1 - \mathbf{h}),$$

where $\odot$ denotes elementwise multiplication. Hence

$$\boxed{\mathbf{g}_{z_1} = \mathbf{g}_h \odot \mathbf{h} \odot (1 - \mathbf{h})} \qquad [H \times 1].$$

Each hidden unit independently receives its accumulated downstream message and gates it by its own sigmoid derivative.

3.6 Step 5: the first affine layer reaches the parameters and input

The first affine layer is

$$\mathbf{z}_1 = W_1 \mathbf{x} + \mathbf{b}_1.$$

Applying the same linear-layer rule gives

$$\boxed{\frac{\partial \mathcal{L}}{\partial W_1} = \mathbf{g}_{z_1} \mathbf{x}^\top} \qquad [H \times D],$$

$$\boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} = \mathbf{g}_{z_1}} \qquad [H \times 1],$$

and, if a gradient with respect to the input is required,

$$\boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = W_1^\top \mathbf{g}_{z_1}} \qquad [D \times 1].$$

The formulas are identical to those used for the second affine layer. Only the dimensions change.

4 The complete backward pass

The entire derivation can now be written as a short right-to-left program:

$$
\begin{aligned}
g_{z_2} &= \hat{y} - y, \\
\nabla_{W_2}\mathcal{L} &= g_{z_2}\mathbf{h}^\top, & \nabla_{b_2}\mathcal{L} &= g_{z_2}, \\
\mathbf{g}_h &= W_2^\top g_{z_2}, \\
\mathbf{g}_{z_1} &= \mathbf{g}_h \odot \mathbf{h} \odot (1 - \mathbf{h}), \\
\nabla_{W_1}\mathcal{L} &= \mathbf{g}_{z_1}\mathbf{x}^\top, & \nabla_{\mathbf{b}_1}\mathcal{L} &= \mathbf{g}_{z_1}, \\
\mathbf{g}_x &= W_1^\top \mathbf{g}_{z_1}.
\end{aligned}
$$

! What to remember

- **BCE plus sigmoid:** error message $\hat{y} - y$.
- **Sigmoid:** multiply elementwise by $h(1-h)$.
- **Linear layer:** transpose to inputs, outer product to weights, copy to bias.
- **Several downstream paths:** add their gradient messages before applying the local derivative.

5 Shape audit

Quantity	Shape	Backward expression
g_{z_2}	scalar	$\hat{y} - y$
$\nabla_{W_2}\mathcal{L}$	$1 \times H$	$g_{z_2}\mathbf{h}^\top$
$\nabla_{b_2}\mathcal{L}$	scalar	g_{z_2}
$\mathbf{g}_h$	$H \times 1$	$W_2^\top g_{z_2}$
$\mathbf{g}_{z_1}$	$H \times 1$	$\mathbf{g}_h \odot \mathbf{h} \odot (1 - \mathbf{h})$
$\nabla_{W_1}\mathcal{L}$	$H \times D$	$\mathbf{g}_{z_1}\mathbf{x}^\top$
$\nabla_{\mathbf{b}_1}\mathcal{L}$	$H \times 1$	$\mathbf{g}_{z_1}$
$\mathbf{g}_x$	$D \times 1$	$W_1^\top \mathbf{g}_{z_1}$

Every parameter gradient has exactly the same shape as its parameter. This is one of the fastest checks for a transposition error.

6 Mini-batches

For a batch of N examples, a common row-major convention is

$$X \in \mathbb{R}^{N\times D}, \qquad Z_1 = XW_1 + B_1, qquadH = \sigma(Z_1).$$

With this convention, the weight gradients become

$$\nabla_{W_2}\mathcal{L} = H^\top G_{Z_2}, \qquad \nabla_{W_1}\mathcal{L} = X^\top G_{Z_1},$$

and bias gradients sum over the batch dimension:

$$\nabla_{b_2}\mathcal{L} = \sum_{n=1}^{N}(G_{Z_2})_n, \qquad \nabla_{\mathbf{b}_1}\mathcal{L} = \sum_{n=1}^{N}(G_{Z_1})_{n,:}.$$

If the loss is defined as a batch mean, divide these accumulated parameter gradients by N. The apparent change in matrix order comes only from switching between column-vector and row-major batch conventions; the underlying messages are unchanged.

7 A final self-check

Before trusting an implementation, verify:

1. The backward pass visits blocks in the reverse of the forward order.
2. Every local derivative is evaluated using a stored forward value.
3. Contributions from multiple children are added.
4. Every parameter gradient matches its parameter's shape.
5. The sigmoid–BCE output message is $\widehat{y} - y$.
6. A finite-difference gradient check agrees with the analytic gradient on a tiny network.

Backpropagation is not one enormous derivative. It is a disciplined protocol for passing, accumulating, and locally transforming small derivative messages.